

Parcial Backend de Aplicaciones 2026

Introducción

El contexto de este parcial es el de generación de liquidaciones de tarjetas de crédito. Se entrega, junto a este enunciado, los scripts de creación de una base de datos reducida para este fin, como así también un script con datos iniciales. Estos archivos son **schema.sql** (estructura) y **data.sql** (datos).

El parcial consta de dos partes: una parte de *preparación*, que los alumnos desarrollan libremente en sus casas (con las asistencias que deseen) y una parte *presencial* en la que deben desarrollar en la Universidad, de manera presencial y con acceso únicamente a las Fichas de Clase.

Estructura de la base de datos

La base de datos se compone de las siguientes tablas:

COTIZACIONES

Contiene una fila por cada tasa de cambio entre el Peso Argentino y otra moneda. Por ejemplo, una fila con Moneda = USD y TASA_CAMBIO = 1.400 significa que un USD vale ARS 1.400.

TARJETAS

Contiene una fila por cada tarjeta de crédito.

Columnas

- ID: Auto incremental con el identificador
- NUMERO: Número de tarjeta como figura en el plástico
- TITULAR: Titular de la tarjeta como figura en el plástico
- LIMITE_CREDITO: El límite de crédito de la tarjeta (Cuánto tiene disponible para consumos por mes)

CONSUMOS

Contiene una fila por cada consumo realizado. Este consumo está asociado a una tarjeta de crédito. El consumo está asociado.

Columnas

- ID: Auto incremental con el identificador
- ID_TARJETA: El ID (no el número) de la tarjeta de crédito asociada al consumo
- MONTO: El monto del consumo
- DIA: El día del mes cuando se realizó el consumo
- MES: El mes de realización del consumo
- ANIO: El año de realización del consumo
- RUBRO: El rubro al que pertenece el consumo (si es supermercado, indumentaria, etc.)
- MONEDA: El código de moneda del consumo (ARS, USD, EUR, etc.). Para fines del parcial, no importa el País de consumo, solamente la moneda.

LIQUIDACIONES

Contiene una fila por cada liquidación realizada. Una liquidación es el resumen de lo que debe pagar el dueño de la tarjeta. Nosotros vamos a considerar mes completo (del primer al último día de cada mes).

Preparación

Para esta etapa, los alumnos deben generar un proyecto Maven con las dependencias necesarias para trabajar con JPA (y lombok si así lo desean), pero *no* debe ser un proyecto SpringBoot. Pueden tomar como base Java 17, Hibernate 6.4.4 y JUnit (que debe estar presente) 6.

Para esta etapa los alumnos deben preparar:

- Todo el código y configuraciones necesarias para poder trabajar con las entidades: **Tarjeta**, **Consumo**, **Moneda** y **Liquidacion**. Este código debe estar previsto para trabajar de acuerdo a la especificación JPA, mapeando correctamente entidades y sus relaciones.
- Todas las configuraciones necesarias para que se ejecuten, al inicio del programa, la creación de la base de datos y la carga de datos iniciales.
- El código de acceso a datos que permita realizar la obtención de todos los objetos y las operaciones CRUD sobre todas las entidades, además de las siguientes consultas:
 - Consumos de una tarjeta que correspondan a un año y un mes específico.
 - Las tarjetas que *no* tienen liquidación para un año/mes específico.
 - La liquidación de una tarjeta para un año/mes específico, buscando por *número de tarjeta, año y mes*.

Es importante que la base de datos se recree en cada inicio de la aplicación. Cada vez que la aplicación inicie, la base de datos debe quedar con la estructura creada (de acuerdo al archivo **schema.sql**) y **únicamente** con los datos que están en el script adjunto **data.sql**. En la etapa presencial, **data.sql** podría reemplazarse por uno distinto y sus datos deberían ser importados.

Se puede trabajar con la base de datos en memoria o en un archivo, siempre y cuando se cumpla lo ya mencionado de que, en cada arranque de la aplicación, la base de datos se recree.

Una manera de lograr esto es mediante las siguientes propiedades en **persistence.xml** (y los archivos .sql en la carpeta **resources**):

```
<property name="jakarta.persistence.schema-  
generation.database.action" value="drop-and-create"/>  
<property name="jakarta.persistence.schema-generation.create-  
source" value="script"/>  
<property name="jakarta.persistence.schema-generation.create-  
script-source" value="schema.sql"/>  
<property name="jakarta.persistence.sql-load-script-source"  
value="data.sql"/>
```

No es obligatorio realizar la creación de la estructura y la carga de datos de esta manera, los alumnos pueden optar por el mecanismo que consideren más conveniente.

El desarrollo que realicen como parte de esta etapa de preparación deberán tenerlo disponible en el momento de la etapa presencial. Pueden subirlo a su repositorio personal de la facultad o tenerlo disponible de otra forma, pero **deben estar preparados para desarrollar el examen en las computadoras de la Universidad**, en caso de que así se requiera.

Etapa Presencial

Se entrega, junto a este enunciado, una serie de clases e interfaces que el alumno debe agregar en su proyecto, respetando la estructura de paquetes.

El archivo comprimido contiene:

- **pom.xml**: Solamente de ejemplo, no es obligatorio utilizarlo
- **Carpeta resources**:
 - **liquidaciones.csv**: necesario para el desarrollo del parcia.
 - *persistence.xml* (de ejemplo).
 - *data.sql* y *schema.sql* (los mismos entregados en el pre-enunciado).
- **Paquetes en la carpeta main/java**:
 - **servicios**: contiene la interfaz que tienen que implementar
 - **excepciones**: contiene una Excepción que deben utilizar
 - **dto**: Contiene la clase DTO que deben utilizar como resultado de las liquidaciones solicitadas
 - **modelo**: Clase ItemLiquidacion que pueden utilizar si les resulta de utilidad, o pueden ignorarla.
- **Tests unitarios, carpeta test/java**: Contiene una clase de Test para realizar las pruebas unitarias de la implementación que realicen los alumnos como parte del parcial.

Liquidaciones

El concepto de liquidación, en el contexto de este parcial, es el resumen de lo que el titular de la tarjeta debe pagar para un año y mes dado. Básicamente, será la suma de todos los consumos realizados (más sus impuestos) y descontando los descuentos que existan.

Debe tenerse en cuenta que la liquidación se expresará siempre en ARS. La tasa de cambio entre ARS y las demás monedas está almacenada en la tabla COTIZACIONES.

Las reglas serán las siguientes:

Descuentos

Moneda ARS

- COMBUSTIBLE: 15% con tope de ARS 750 por transacción
- SUPERMERCADOS: 20% con tope de ARS 3000 por transacción
- RESTAURANTES: 25% entre los días 10 y 15 de cada mes (ambos incluidos)

Moneda extranjera (Cualquiera ue no sea ARS)

No hay descuentos

Impuestos

Vamos a considerar que los impuestos se calculan sobre el monto completo. Si un consumo era de \$10.000 y luego del descuento quedó en \$8.000, vamos a calcular el impuesto sobre los \$10.000.

Moneda ARS

- IVA: 21%
- Resolución BDA Nro. 1234/95: 12% para rubro OTROS

Moneda extranjera (Cualquiera ue no sea ARS)

- Impuesto extraordinario: 7.5 %

Requerimientos

1. Crear una clase que implemente la interfaz `LiquidacionService`, de acuerdo a los comentarios que se encuentran en la misma.
2. Asegurarse que esta implementación pase los tests unitarios entregados con el enunciado.

Aclaraciones sobre la estructura de carpetas y agregados

Se entregan la interfaz `LiquidacionService`, la excepción `TarjetaInexistenteException` y el DTO `LiquidacionDTO` para que puedan integrar el código del Pre-Enunciado con los tests unitarios. **Deben** mantener la estructura de paquetes en las clases entregadas, no así en el código del pre-enunciado donde los paquetes pueden mantener el nombre que ya le han asignado.

En la clase `LiquidacionServiceImplTest` hay un método llamado `setUp` y anotado con `@BeforeEach`, ese método se va a ejecutar antes de cada test. Recordando que la base debía recrearse en cada inicio, el método `setUp` levanta la `PersistenceUnit` antes de ejecutar cada test, para que el test se ejecute con los datos tal como están en el script de inicialización. En este método deben instanciar su implementación de `LiquidacionService` junto a sus dependencias. Se debe notar que, ahí mismo, se está obteniendo una instancia del `EntityManager` que será utilizada en las pruebas, es importante que se utilice esa instancia porque sobre ella se inician y confirman las transacciones.

Para ejecutar los tests, el comando es:

```
mvn test
```

Nada evita que puedan hacer pruebas con un método `main` y las salidas que deseen, pero se espera que al ejecutar los tests unitarios, los mismos se ejecuten correctamente.